IMEN891K / IDSC501 / AXGS-501 · DATA ENGINEERING · FALL 2026

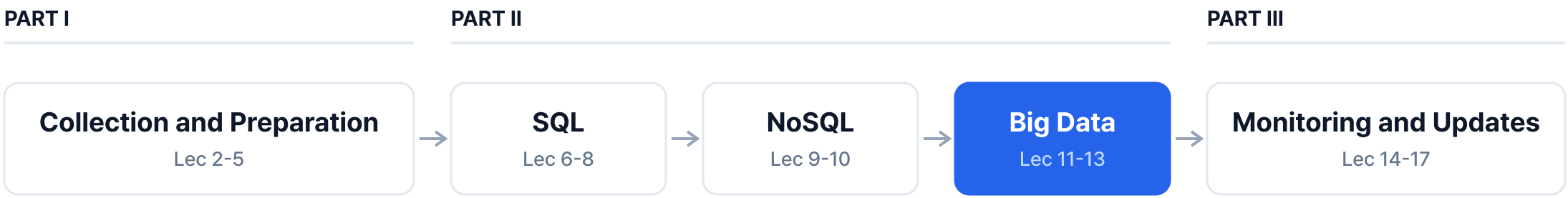
13.

Big Data III: Stream Processing

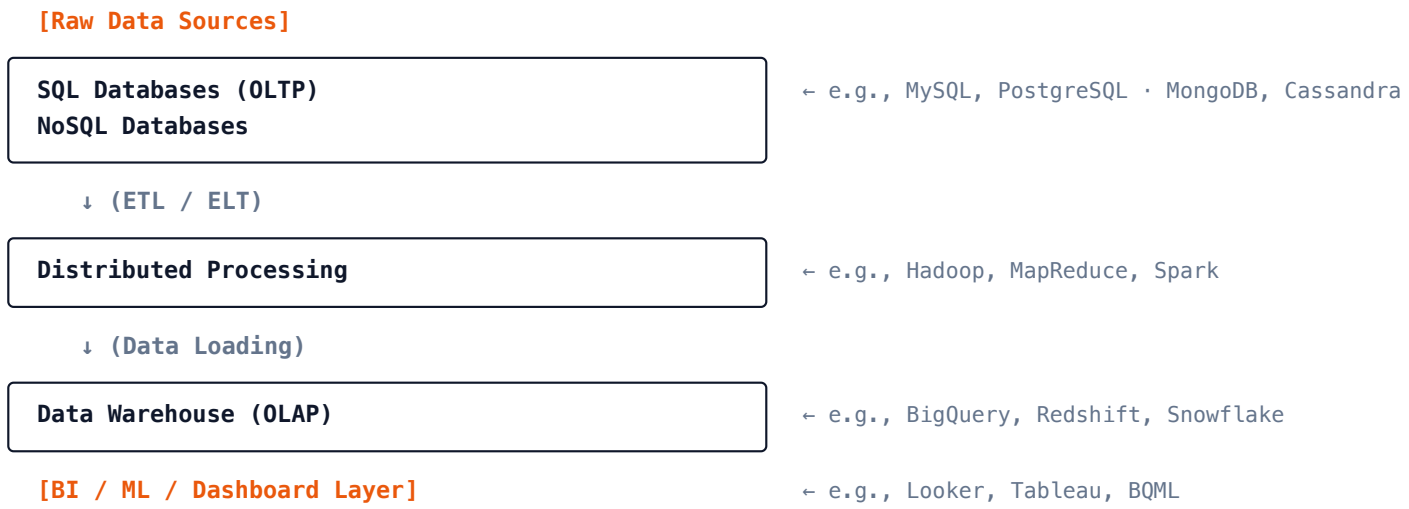
Sangwoo Mo · POSTECH

Industrial & Management Engineering

Where We Are


PART I
PART II
PART III
Collection and Preparation
Lec 2-5
SQL
Lec 6-8
NoSQL
Lec 9-10
Big Data
Lec 11-13
Monitoring and Updates
Lec 14-17

Recap: Where the data has been so far

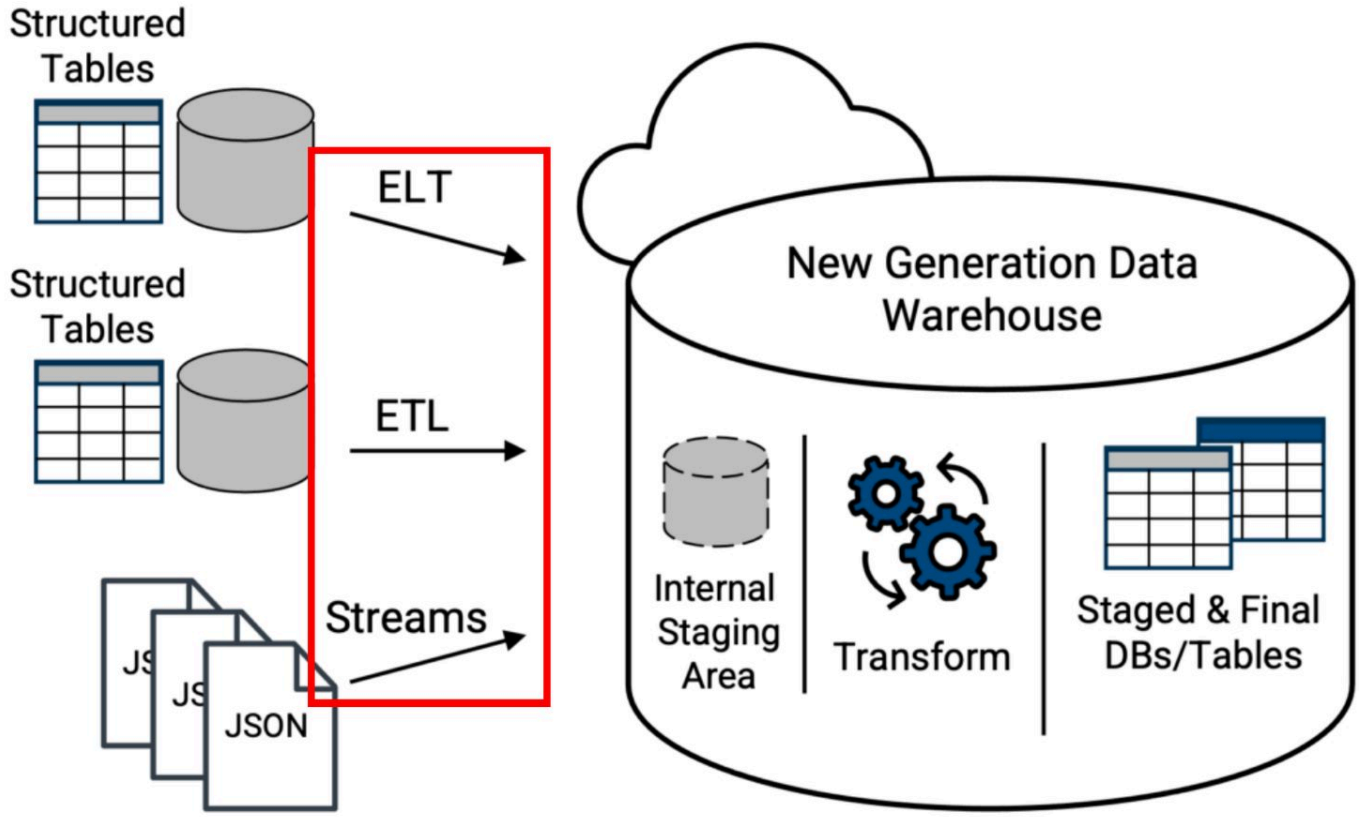


Databases (OLTP) → processing (MapReduce, Spark) → warehouse or lakehouse (OLAP) → dashboards, with Airflow scheduling the steps. Every arrow so far moved data **in batches**.

Outline: Big Data III

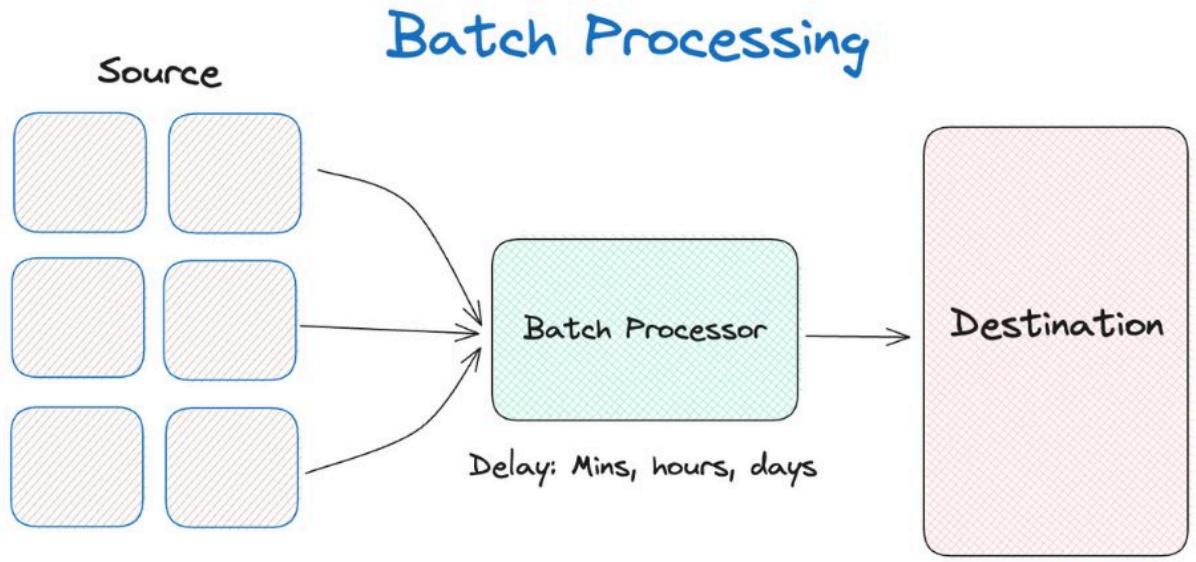
01	**From Batch to Live Events**	handle events as they arrive
02	**Moving Events with Kafka**	store events and deliver them reliably
03	**Processing Event Streams**	connect sources and process live data

Streams in the Data Flow



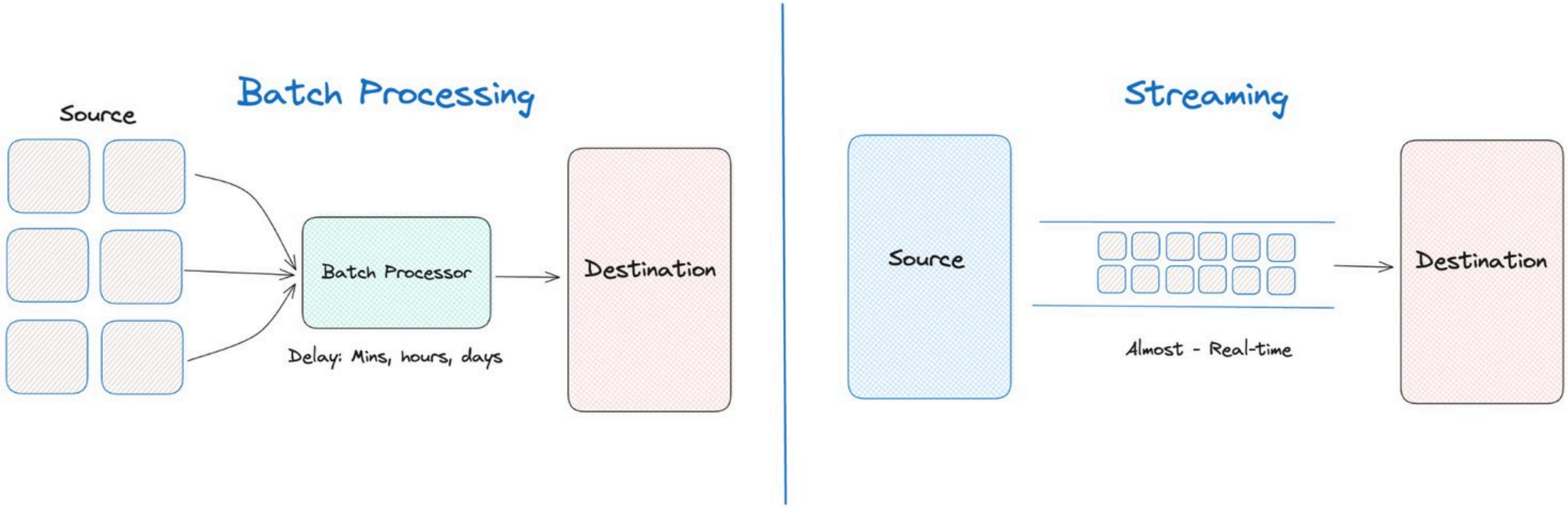
Three arrows in; the third, **Streams**, is today

Batch Processing



Batch: collect, then process, then deliver. **Delay: minutes to days**

Stream Processing



Streaming: each record flows as it happens. **Almost real-time**

From Snapshots to Events

The shift to event-driven systems has already begun...

From a static snapshot...

- ☎ Occasional call to a friend
-  Daily news reports

→

...to a continuous stream of events

- A constant feed about the activities of all your friends
- Real time news feeds, accessible online anytime, anywhere

Why an Event Log

This leads us to ...

- ✅ **A single platform** to connect everyone to every event
- ✅ **Real-time streams** of events
- ✅ **All events stored** for historical view

Case study: Coupang Eats Data Flow

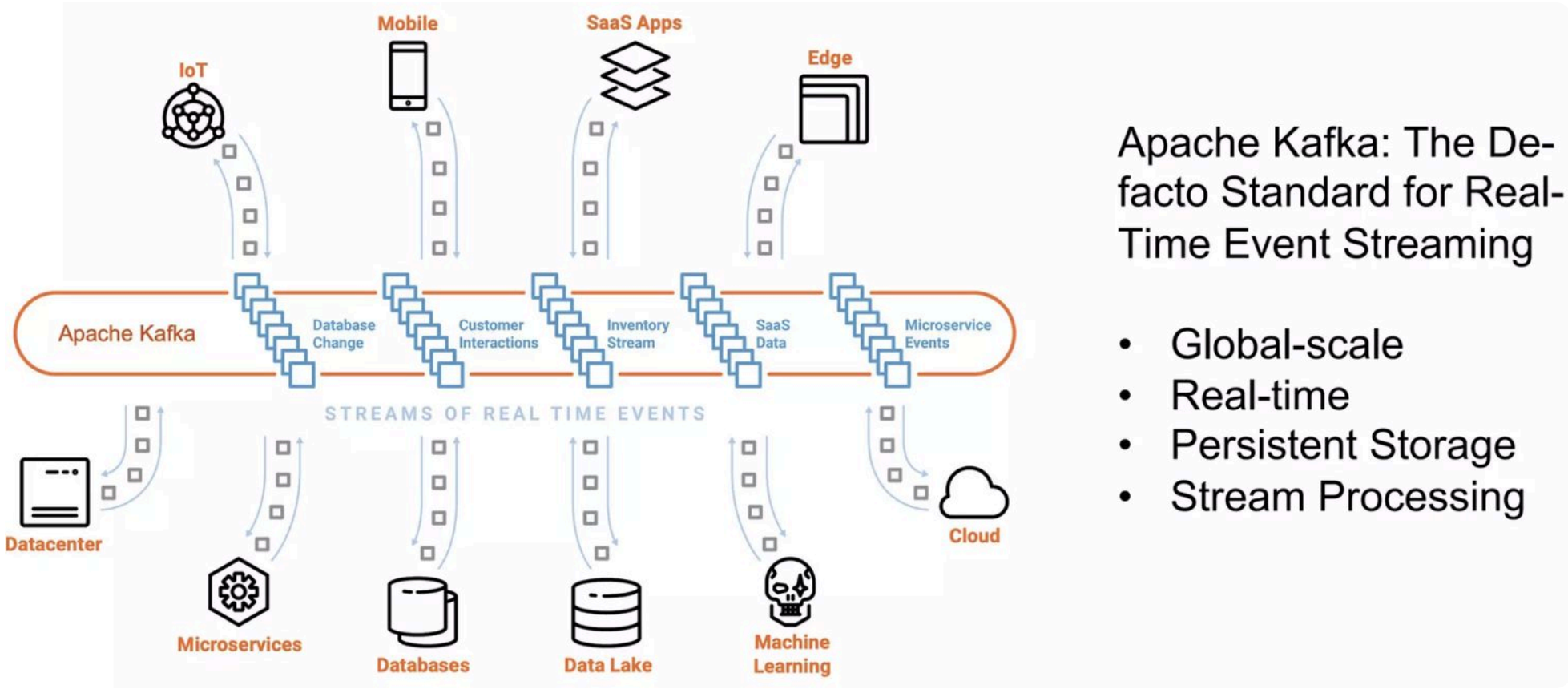
coupang eats data ingestion and pre-processing

Flow	Ingestion	Processing	Data engine
Non-real-time	ETL software · Kafka · cloud storage	Hive, job scheduler	Offline data engine (velocity / relationship)
Near real-time	Kafka	OLAP engine	NRT data engine (velocity)
Pure real-time	Kafka · feature service API	Spark / distributed processing · real-time consumer · Kafka	Pure real-time data engine (velocity / relationship / LBS)

Three latencies, one ingestion bus: **Kafka** appears in every row.

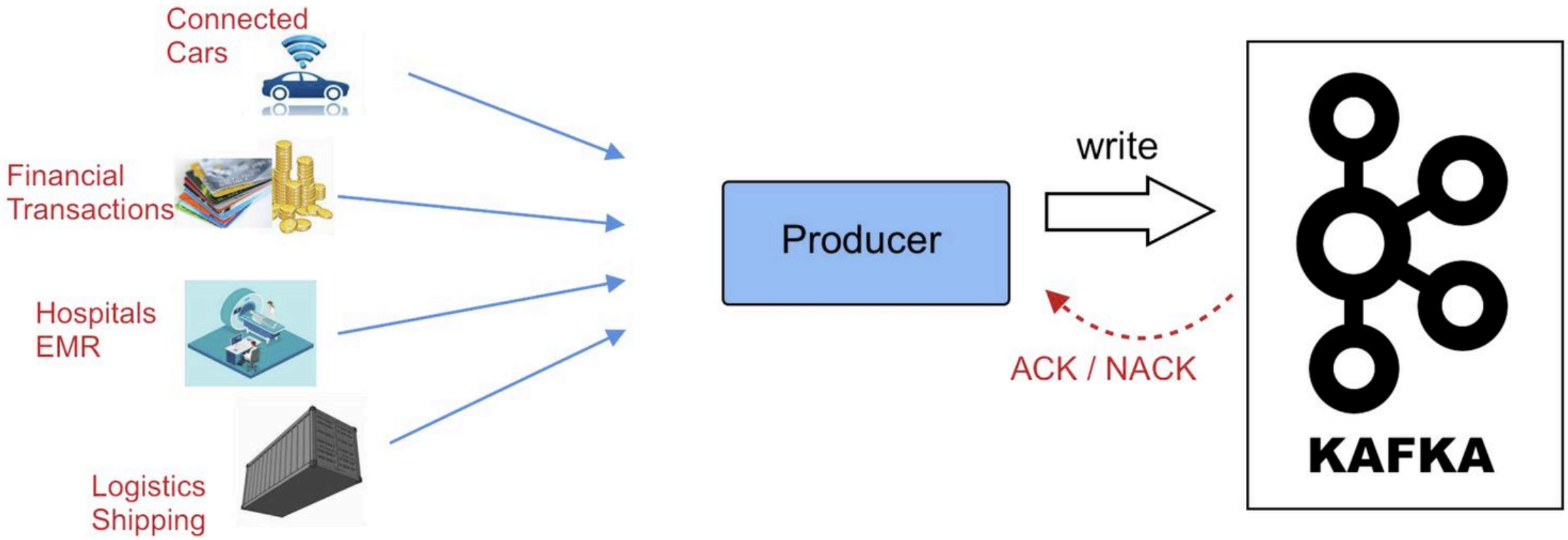
Kafka carries events into batch, near-real-time, and live processing paths

Kafka



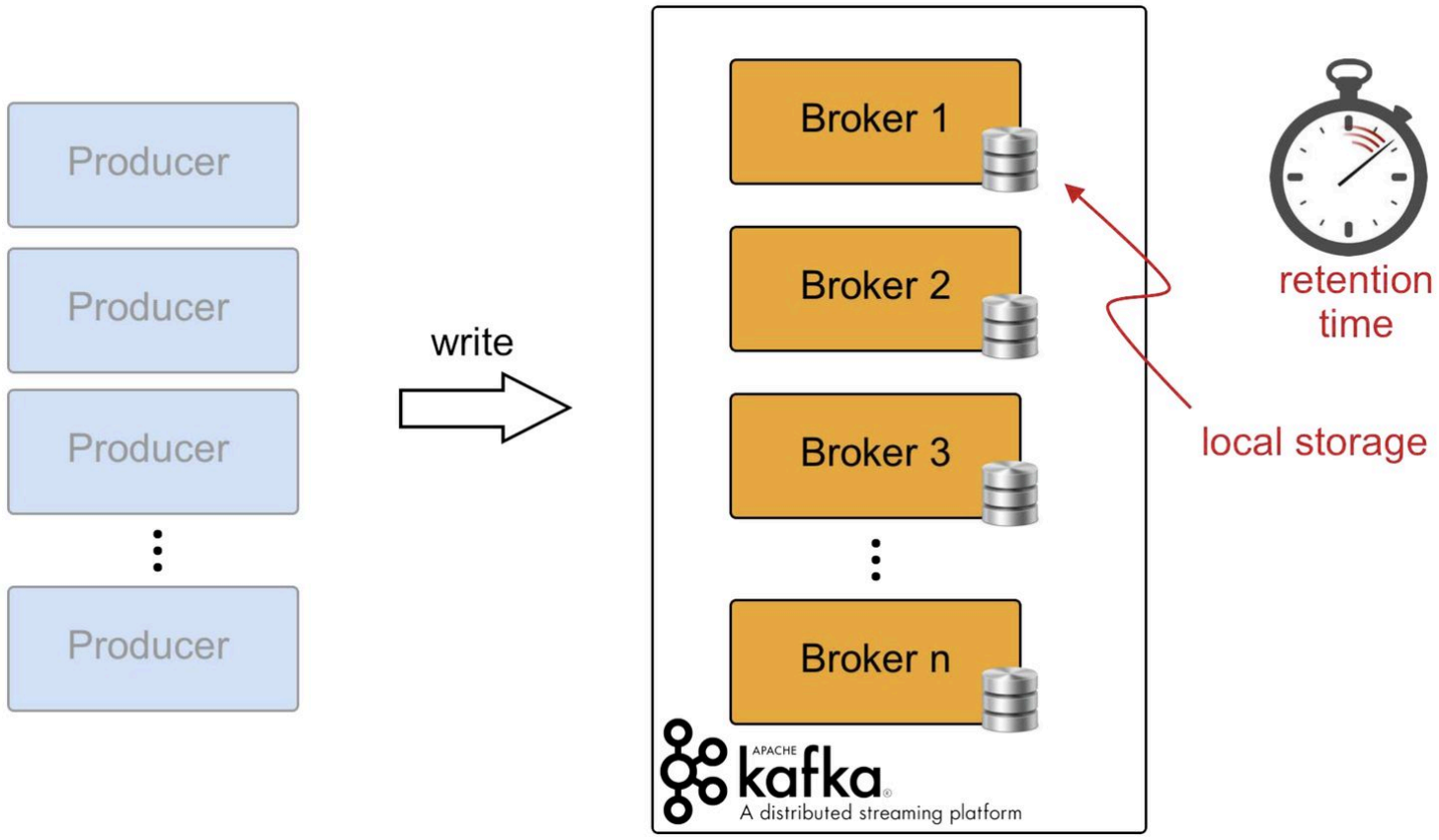
Everything that emits events on top, everything that wants them below, **one log in between**

Kafka – Producers



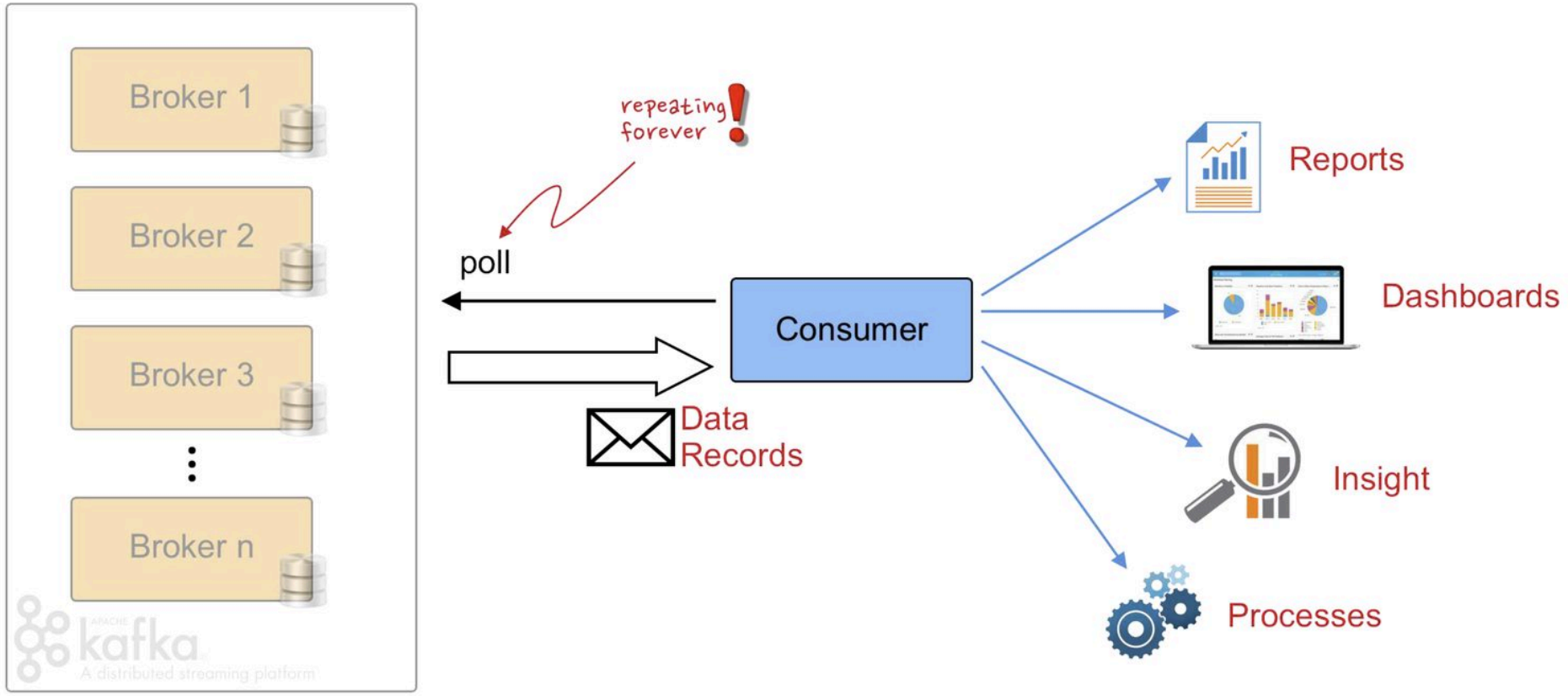
Producers write and get an acknowledgement; **they never know the readers**

Kafka – Brokers



Brokers store events on local disk for a **retention time**: a store, not a queue

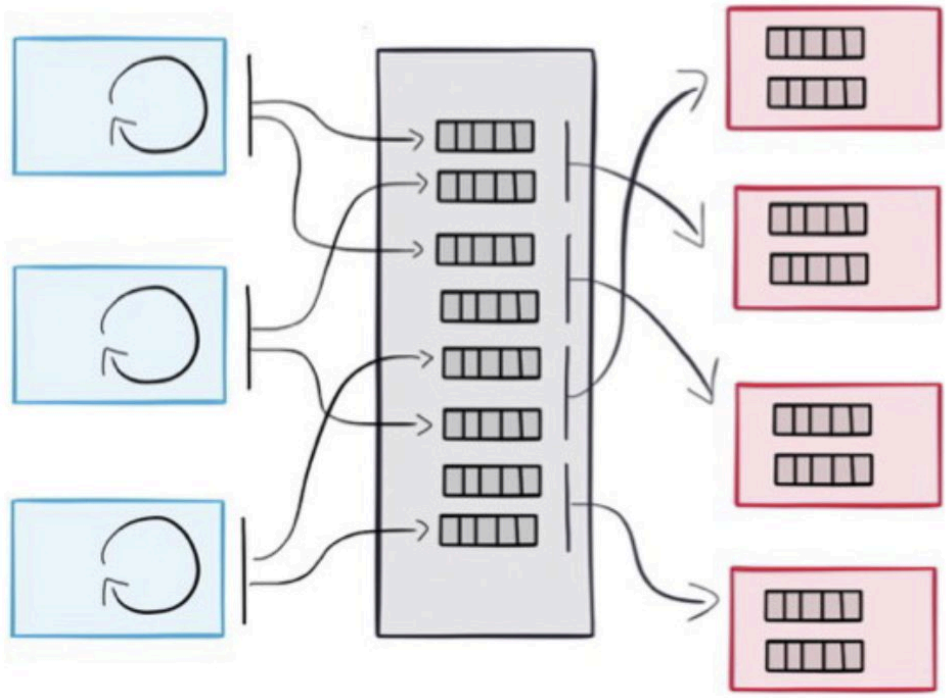
Kafka – Consumers



Consumers **poll**, forever, at their own pace

Decoupling Producers and Consumers

- Producers and Consumers are decoupled
- Slow Consumers do not affect Producers
- Add Consumers without affecting Producers
- Failure of Consumer does not affect System

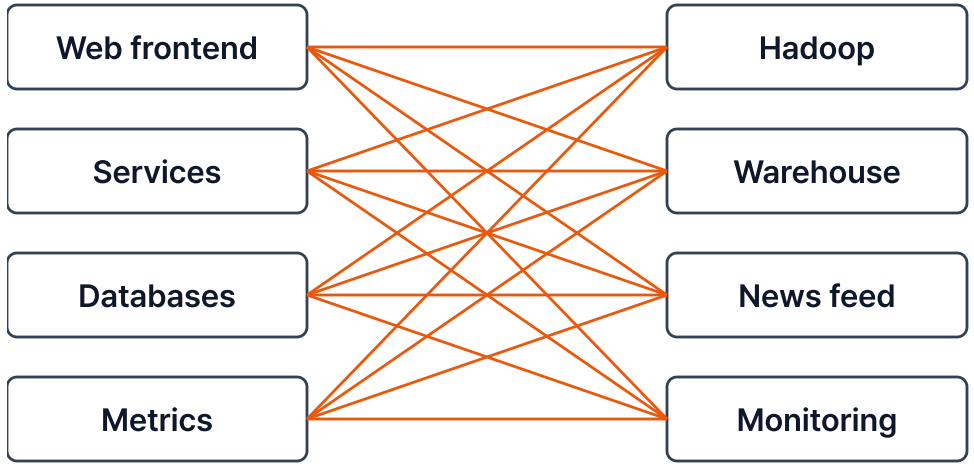


The log in the middle lets each side **fail, slow down or grow** alone

Case study: Kafka at LinkedIn

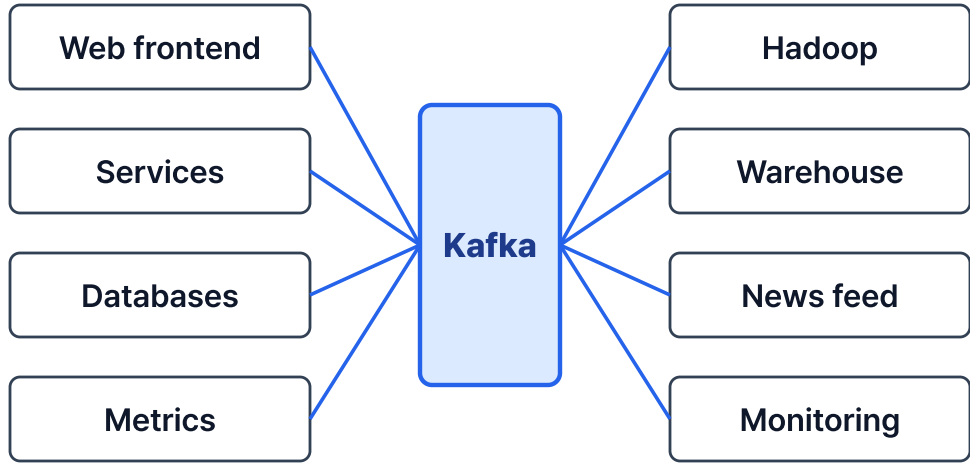
LinkedIn open-sourced Kafka in 2011 to distribute activity and operational events

Illustrative: All-to-All Pipelines



4 × 4 = 16 custom pipelines

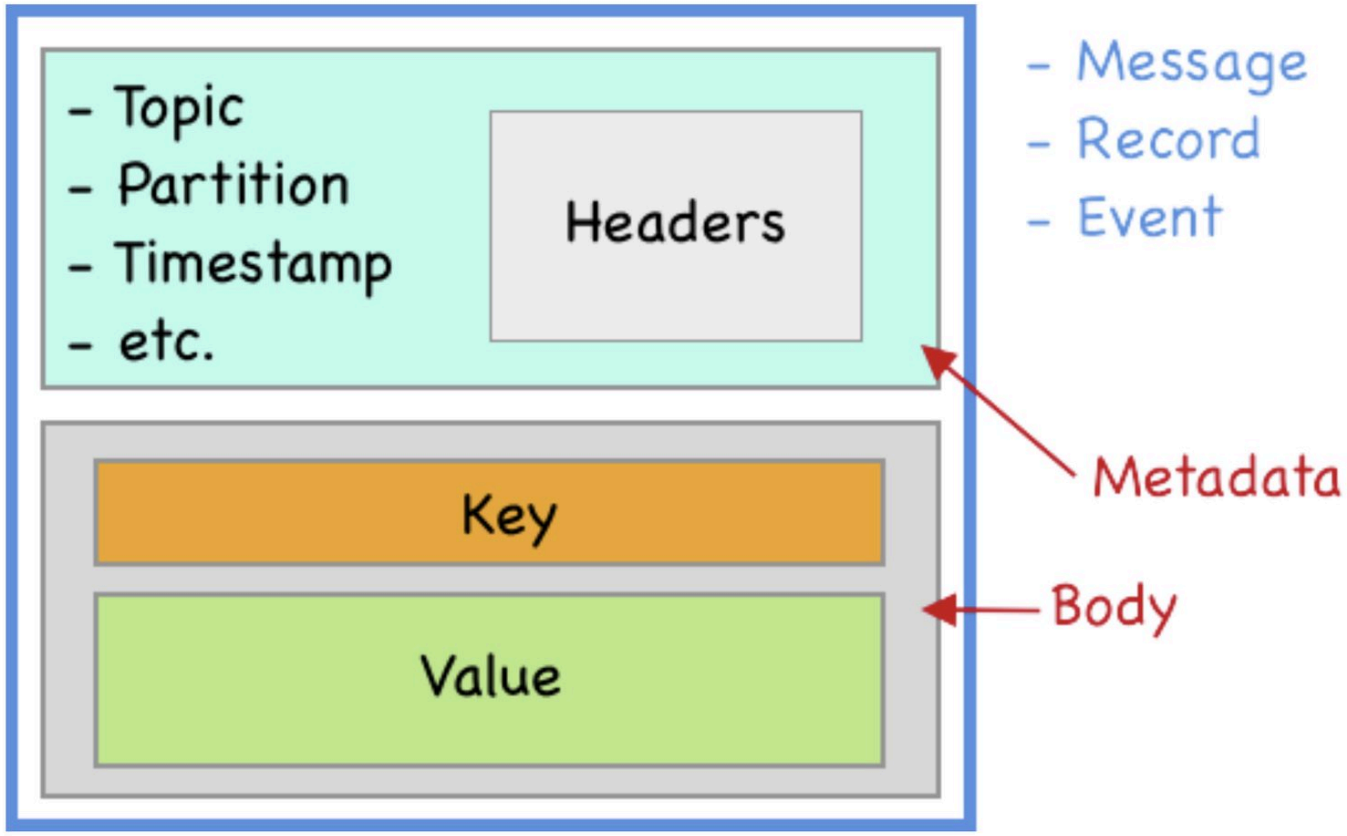
After: One Shared Log



4 + 4 = 8 connections to one log

With all-to-all integration, a shared log reduces endpoint connections from **n×m** to **n+m**

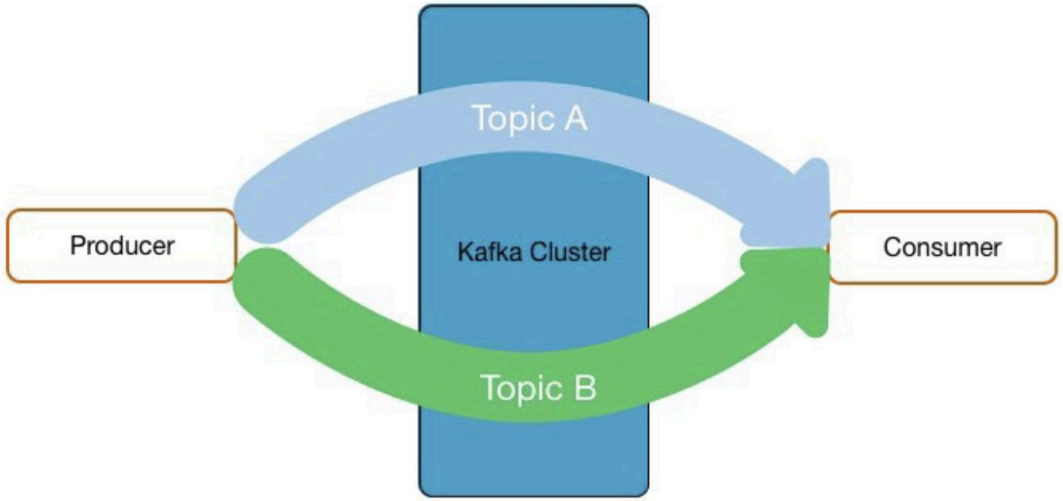
Data Elements



Metadata and optional headers on top; **key and value**, both bytes, as the body

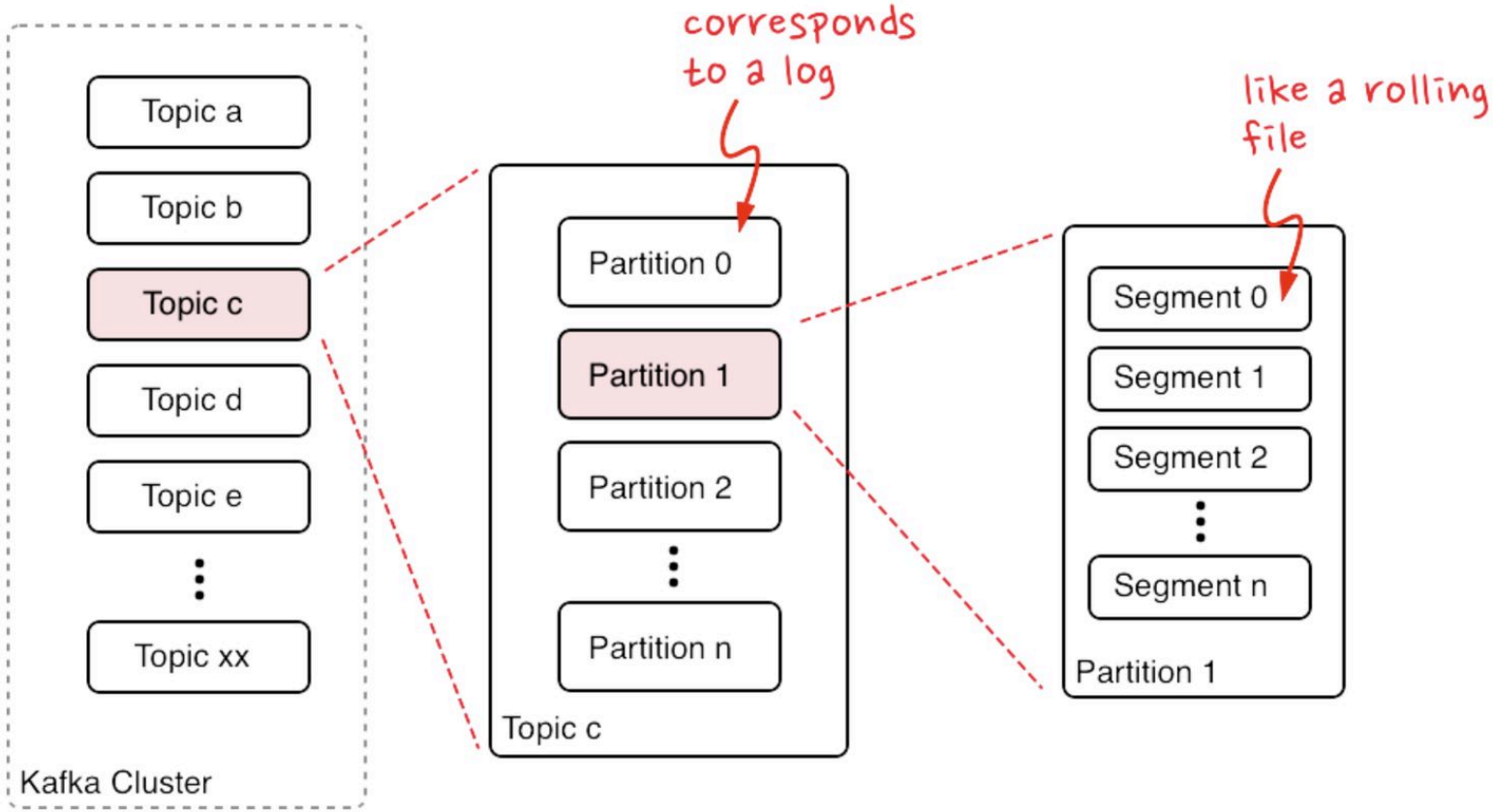
Topics

- **Topics**: streams of “related” messages in Kafka
  - Is a logical representation
  - Categorizes messages into groups
- Developers define topics
- Producer ↔ topic: N to N relation
- Unlimited number of topics



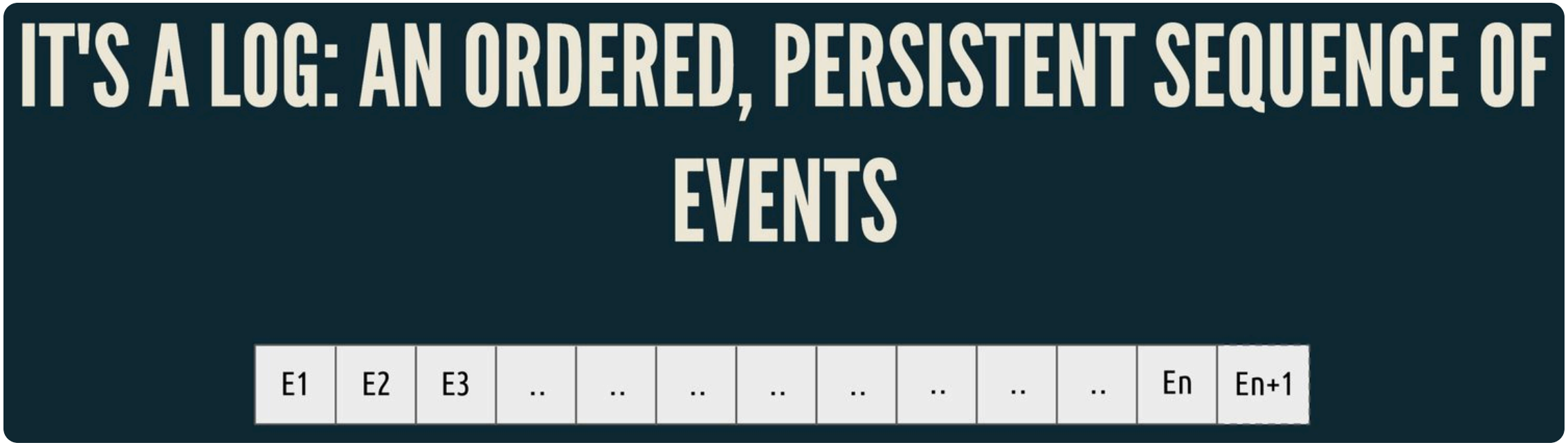
A topic is a **named stream**: the nearest thing to a table

Topic, Partition, Segment



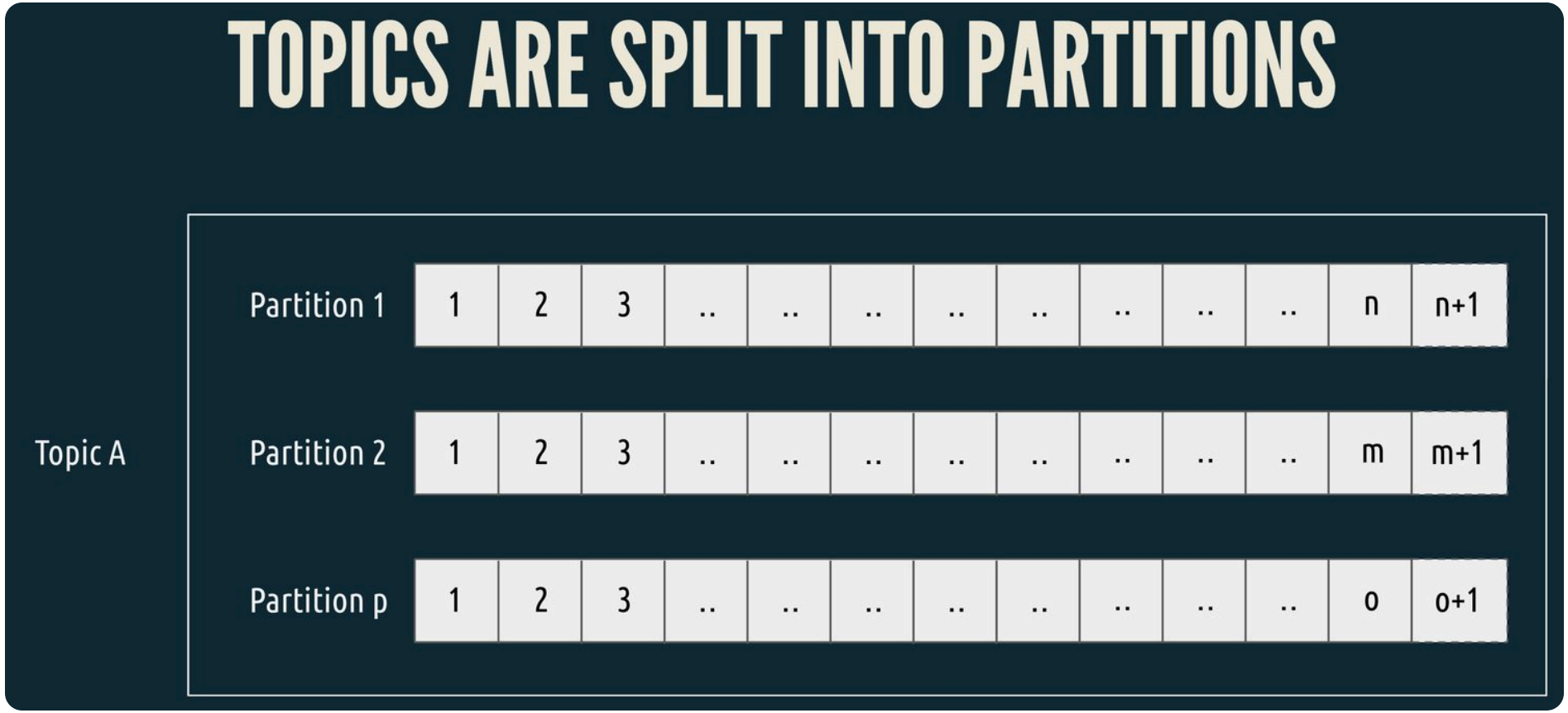
Topic → partitions (each a log) → segments (rolling files on disk)

An Append-Only Log



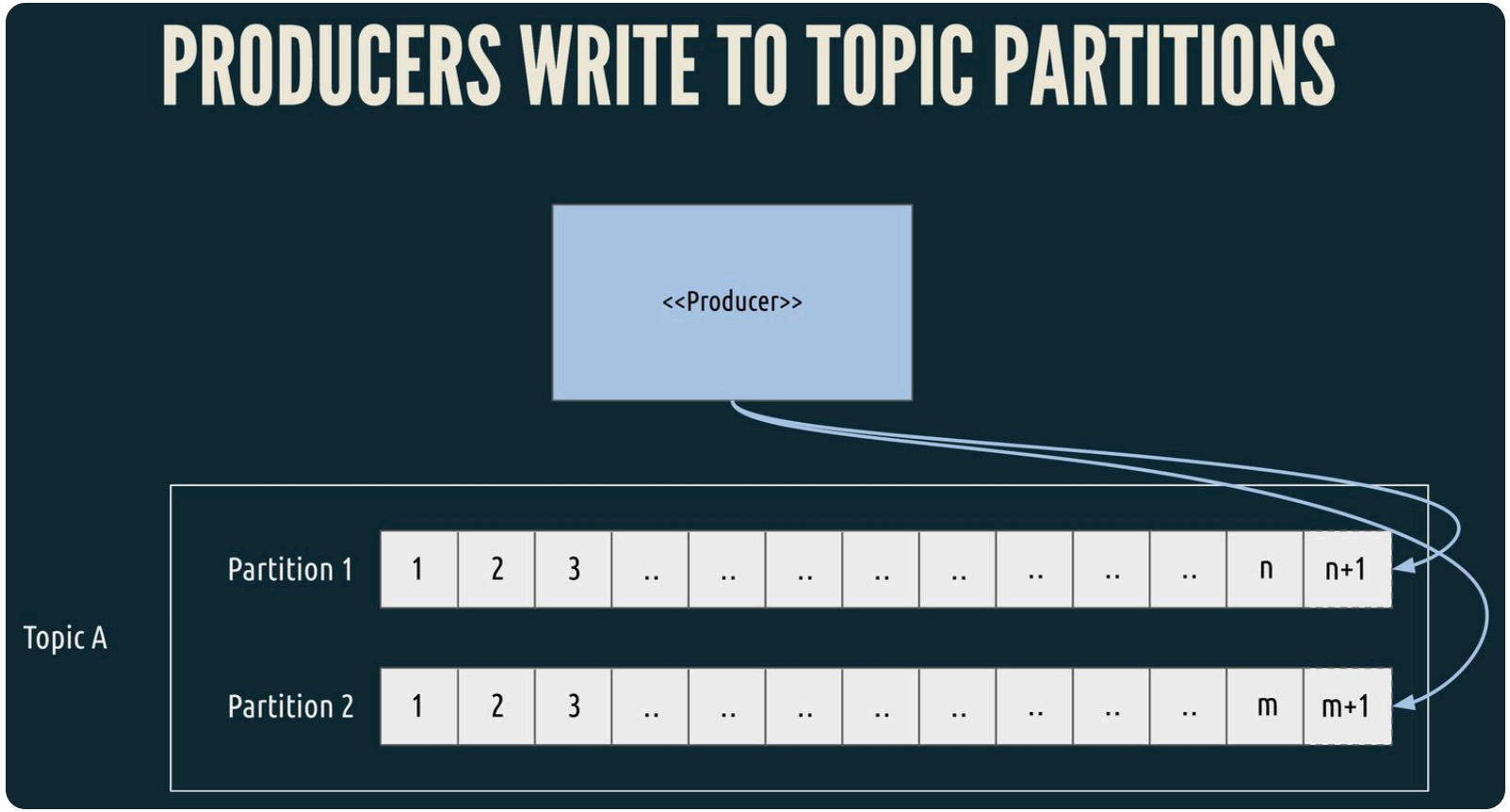
It's a log: an ordered, persistent sequence of events, **append-only**

Order Within a Partition



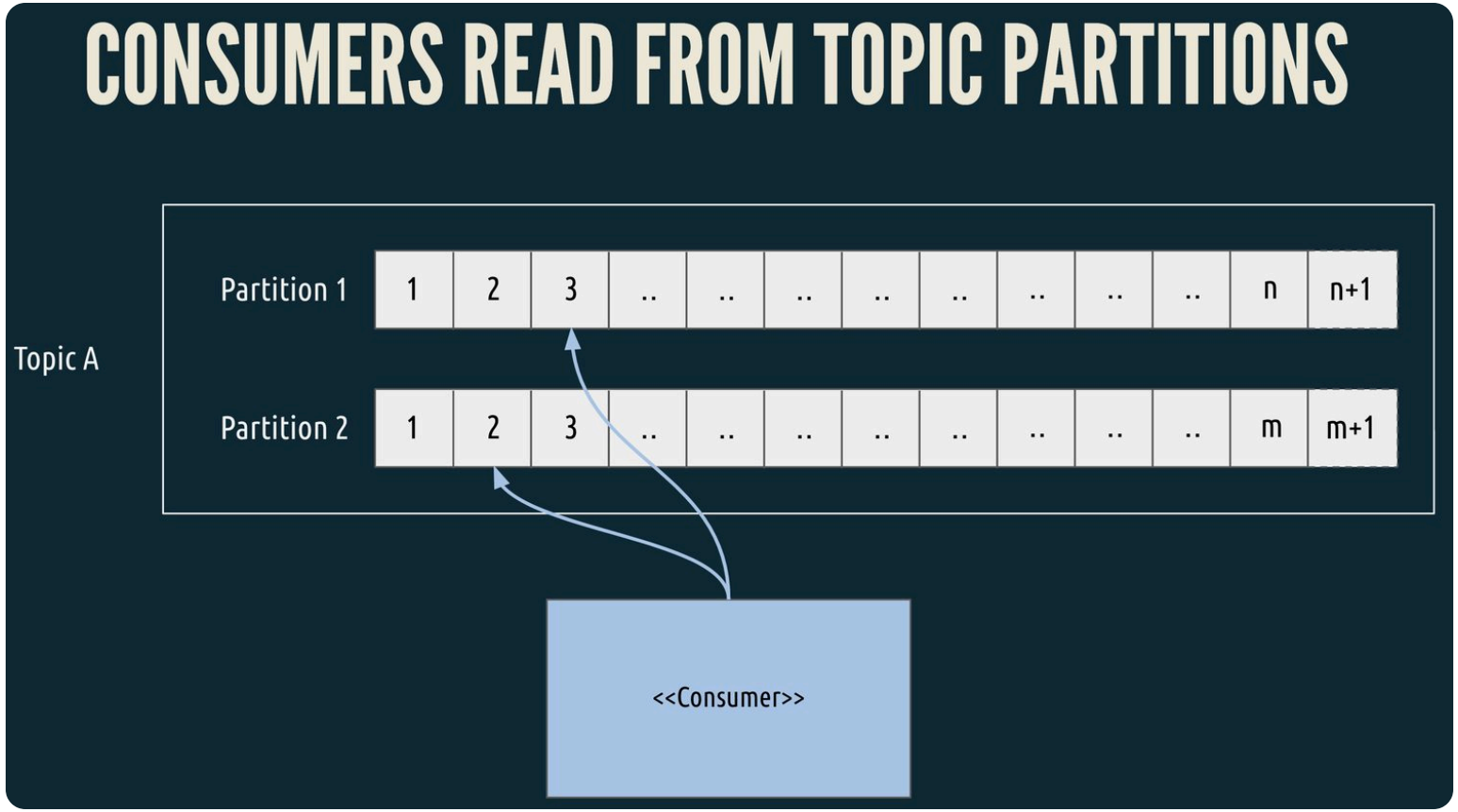
Topics are split into partitions: **order holds within one partition**, not across

Writing to Partitions



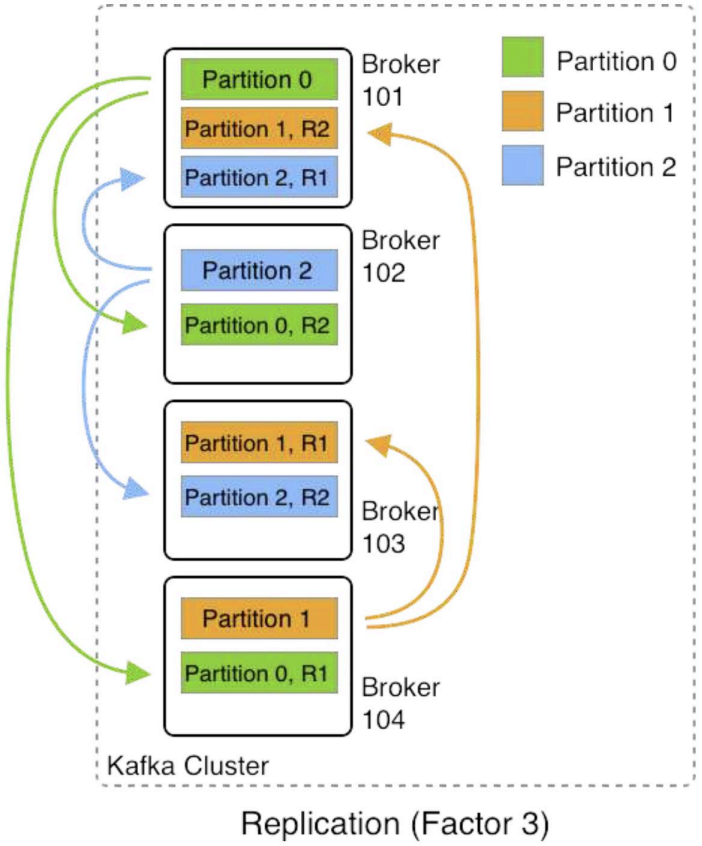
Producers write to topic partitions, **at the end**

Reading Without Deleting



Consumers read from topic partitions; **reading removes nothing**

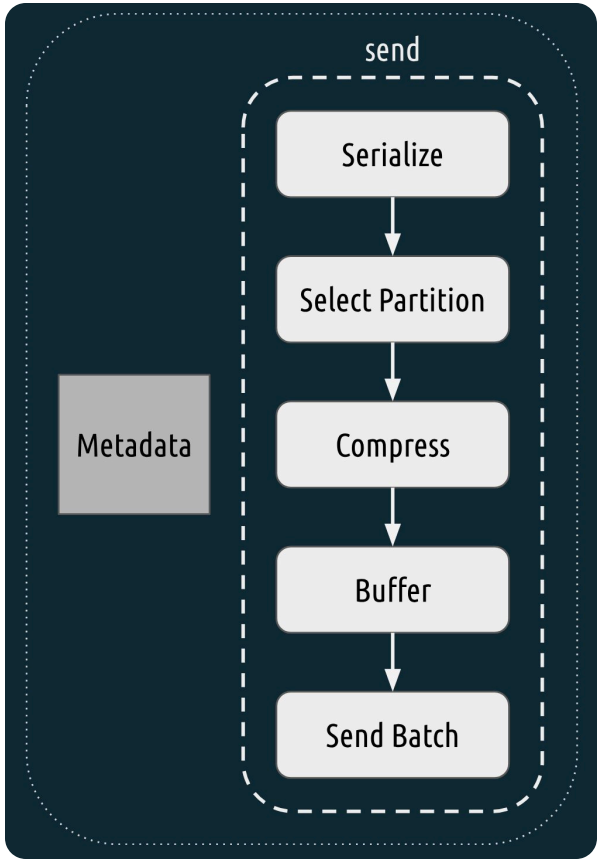
Broker Replication: Fault Tolerance



Replication (Factor 3)

Here each partition has a **leader and two followers** on different brokers

Producer

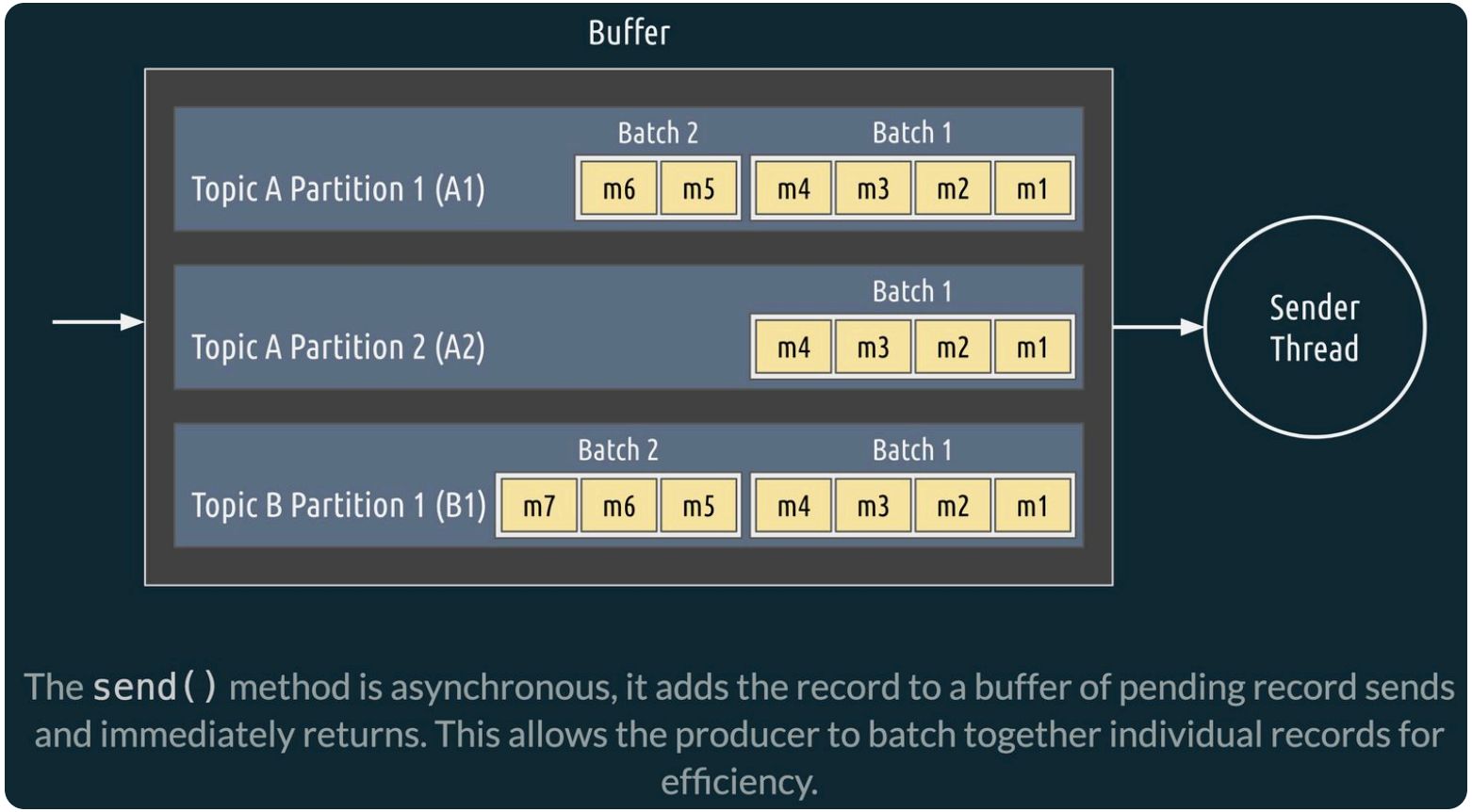


Serialize, pick a partition, compress, buffer, send a batch

Producer – Partitioning Strategy

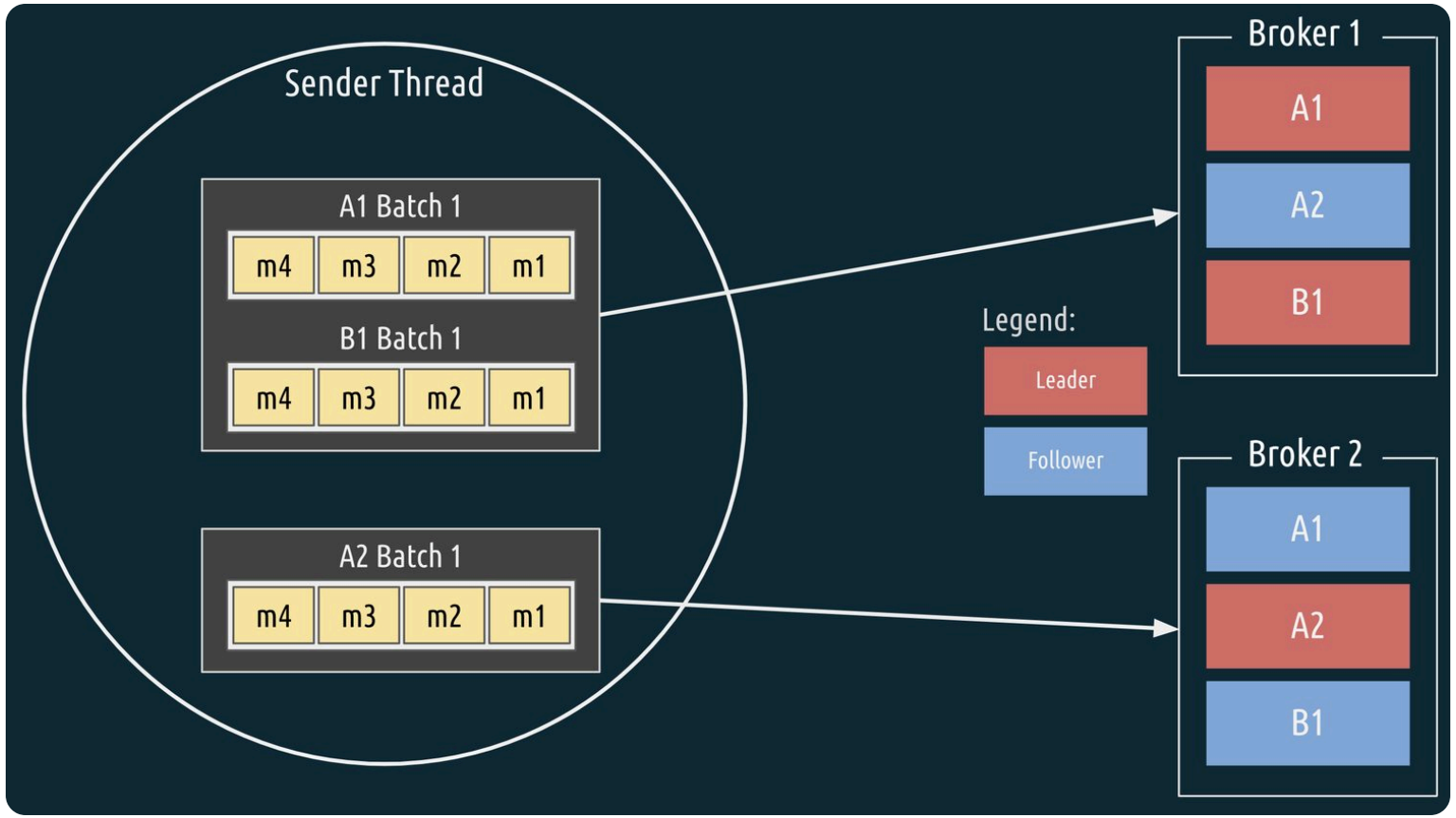
- Producers use a partitioning strategy to assign each message to a partition
- Two purposes:
  - Load balancing
  - Semantic partitioning
- Partitioning strategy specified by producer
  - With a key: `hash(key) % number_of_partitions`
  - No key → spread over partitions (sticky batches since Kafka 2.4)
- Custom partitioner possible

Producer – Accumulate Messages



send() returns at once; records **accumulate per partition** in the buffer

Producer – Send Message Batches

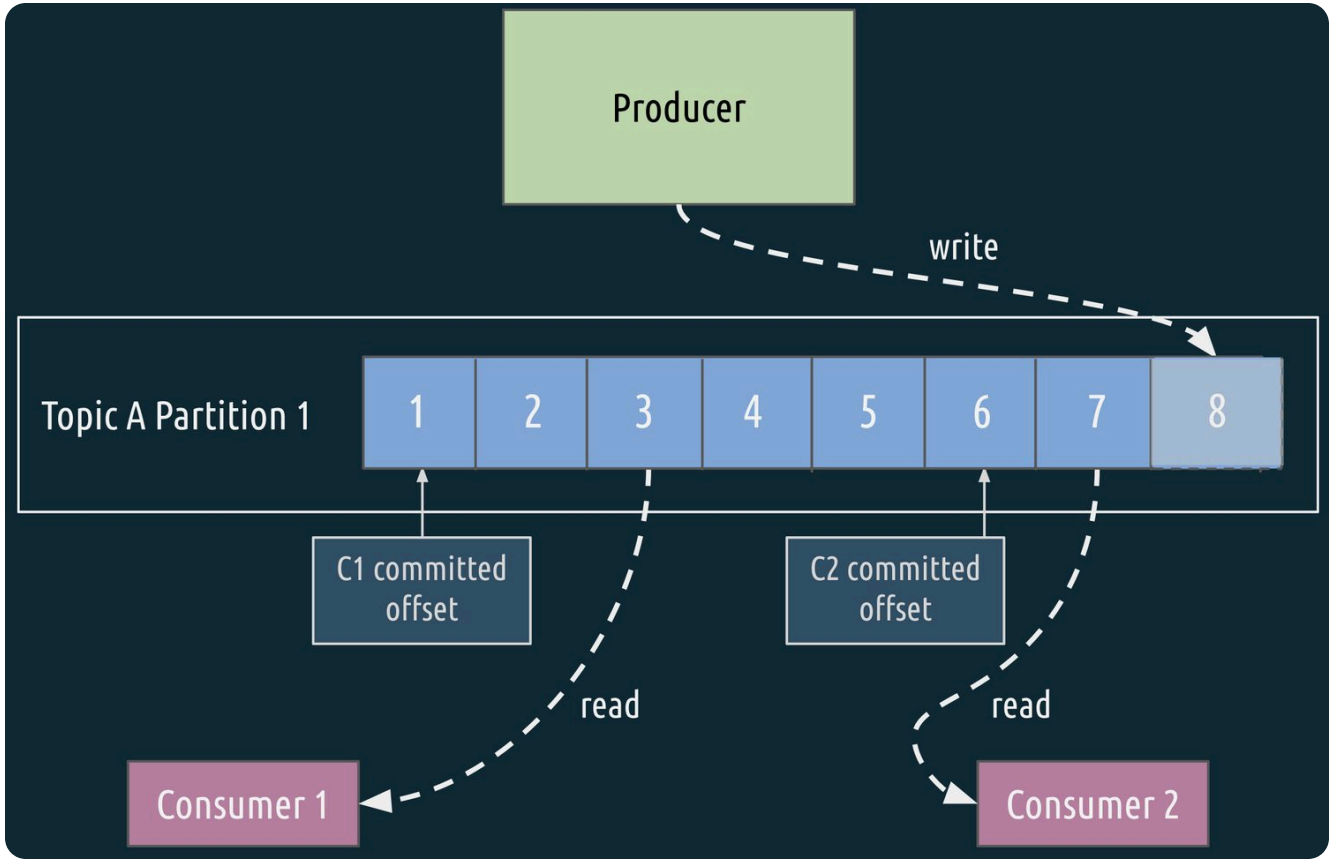


Batches go to each partition's **leader**; one request per broker

Consumer

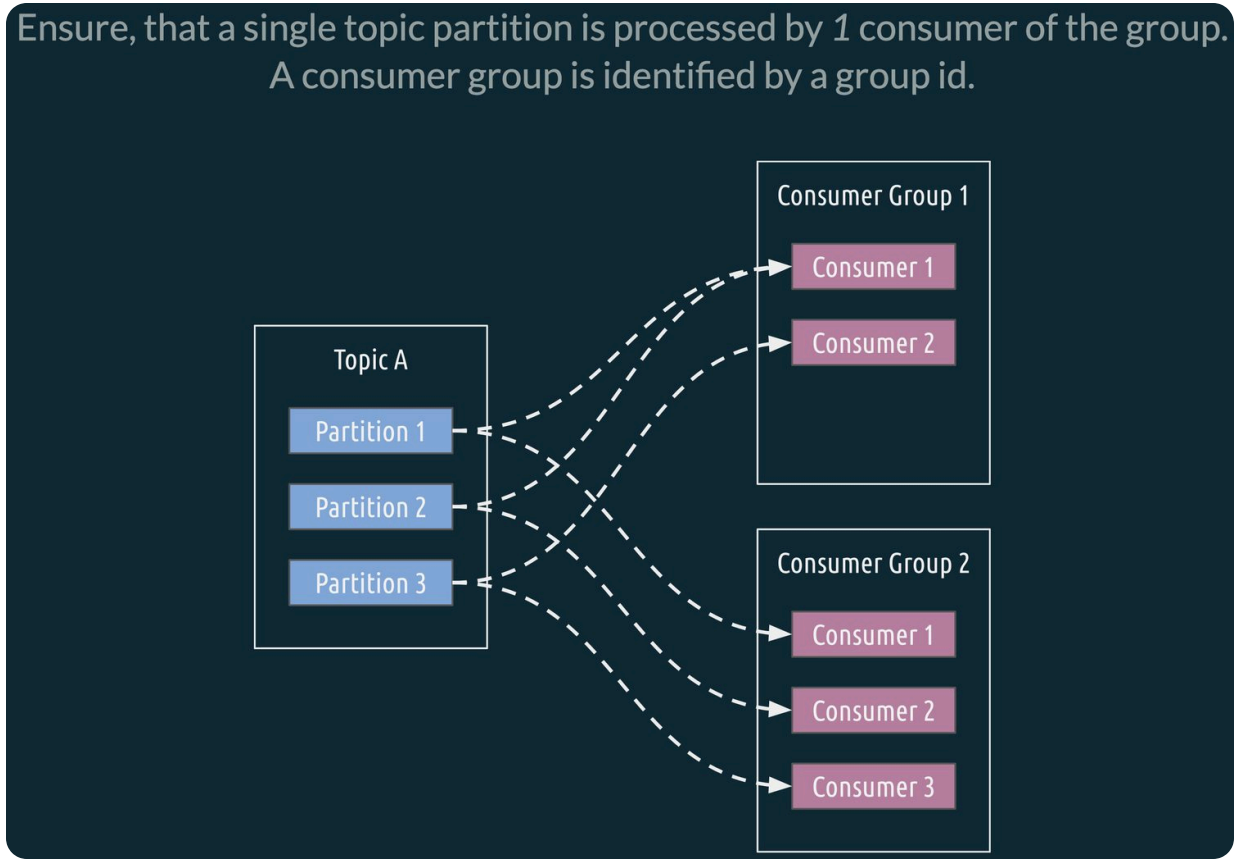
- connect to broker(s) and cache metadata
- subscribe to topic(s)
- join consumer group
- continuously: poll records from broker(s) and commit offsets

Consumer – Offsets



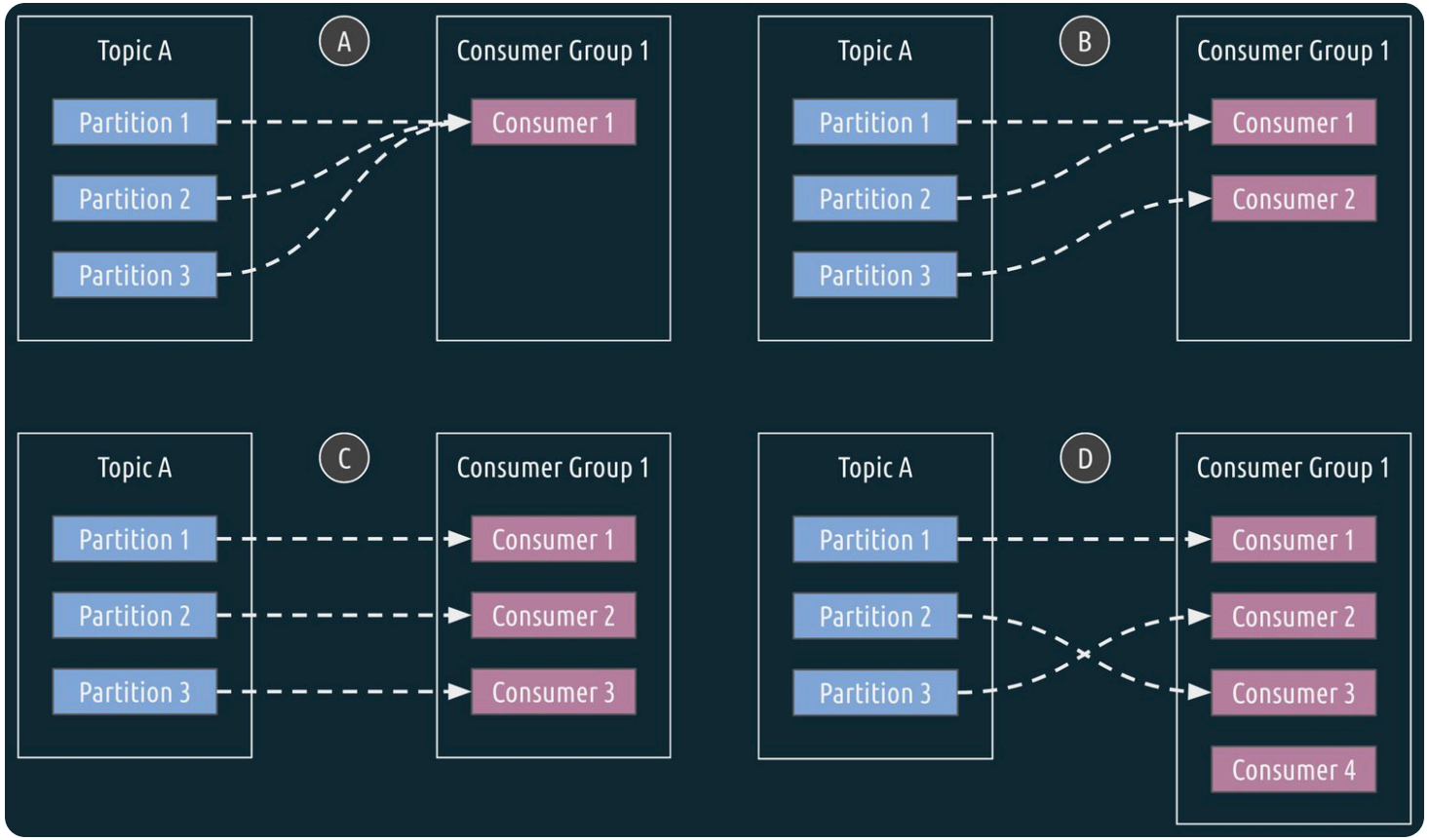
An offset is a position in the log; each consumer keeps **its own**

Consumer – Groups



Within a group a partition has **one reader**; different groups each read everything

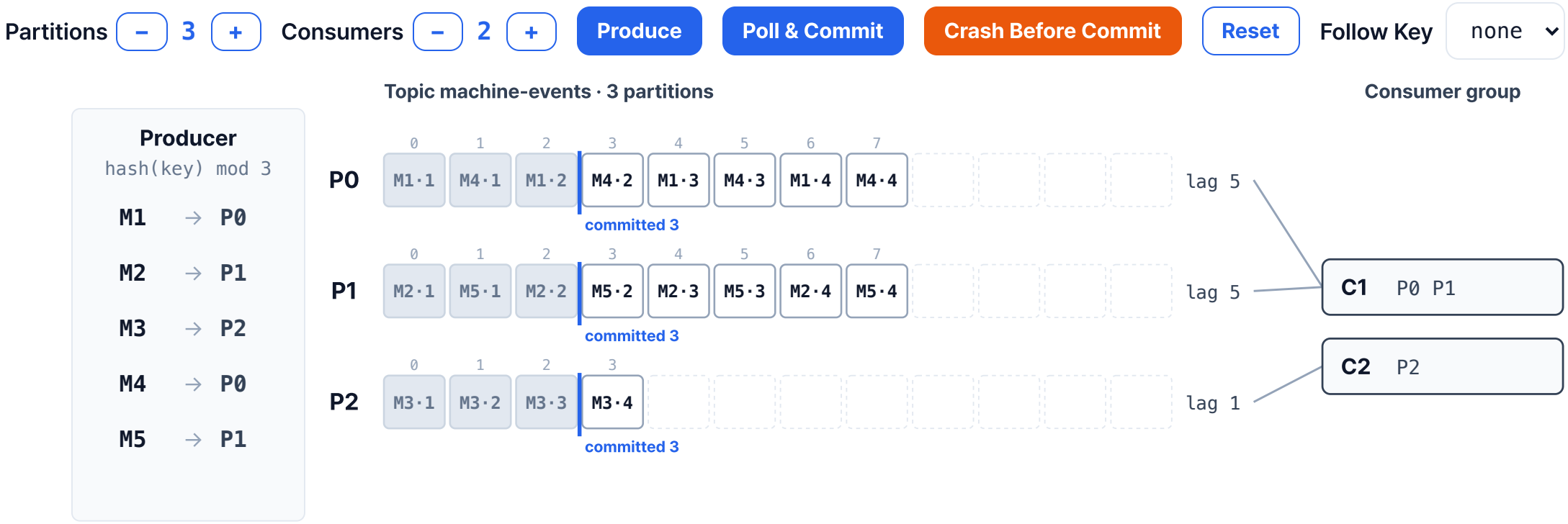
Consumer – Group Rebalancing



Partitions are reassigned as consumers come and go; **partitions cap the parallelism**

Demo: Partitions and Consumer Groups

Five machines send keyed events; change the group, then crash a consumer before it commits



Processed 9 · distinct 9 · **duplicates 0** · total lag 11

Start: 3 partitions, 2 consumers. C1 owns P0 and P1, C2 owns P2. Gray cells are processed and committed

Answer: Partitions and Consumer Groups

Explain the key order, parallelism limit, and replay after a crash

Change	Observation	Reason
Produce for M1	Same partition; increasing offsets	Key mapping is fixed
3 partitions; 4 consumers	One consumer is idle	One owner per partition in this group
Crash before commit; poll again	Previously processed records repeat	Restart from the committed offset

The stored log and the committed offset determine what is read

Caveat: At-Least-Once Duplicates

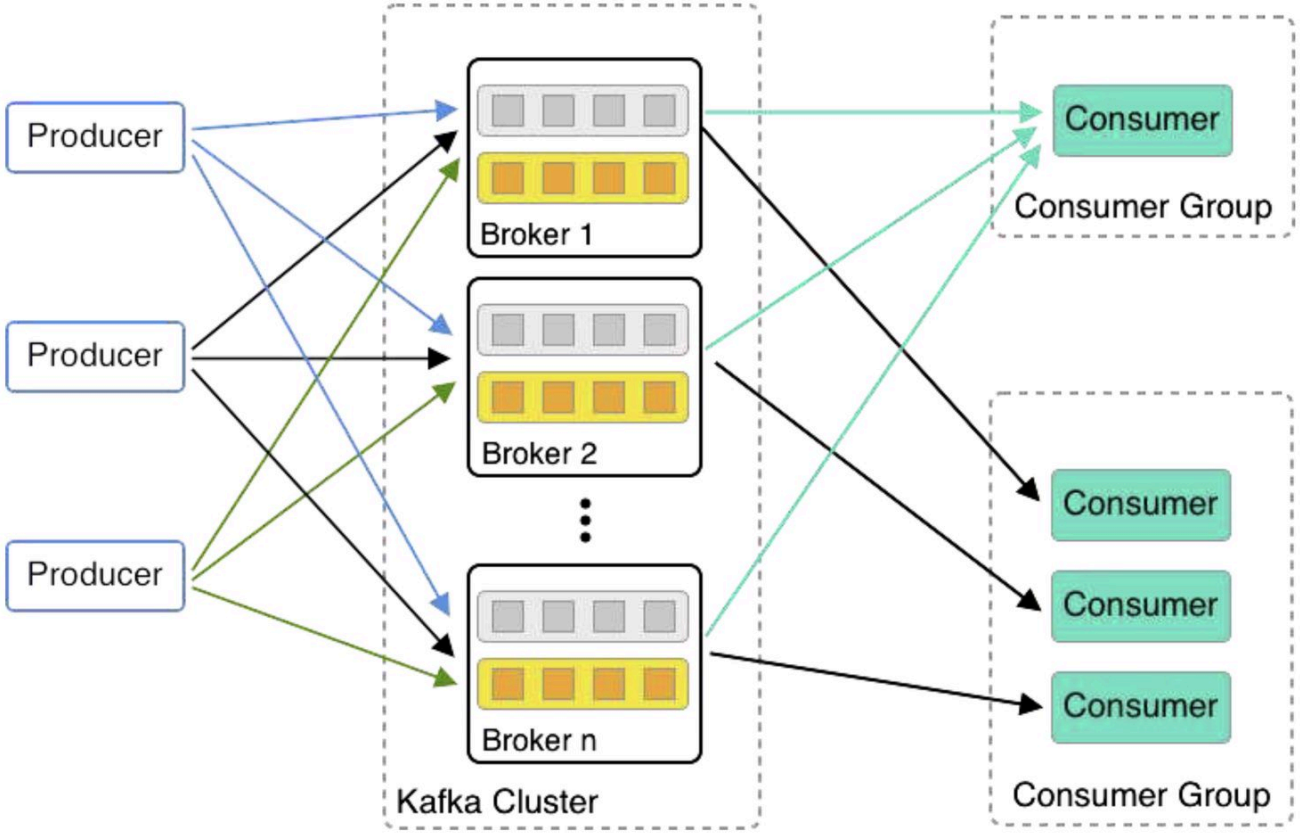
Process, then crash before the commit: the restart reads the same records again



Remedy: deduplicate by event ID, recorded in the same transaction as the side effect

The commit never happened, so offsets **4 to 6 are processed twice**

Conclusion: Scalable Data Pipeline

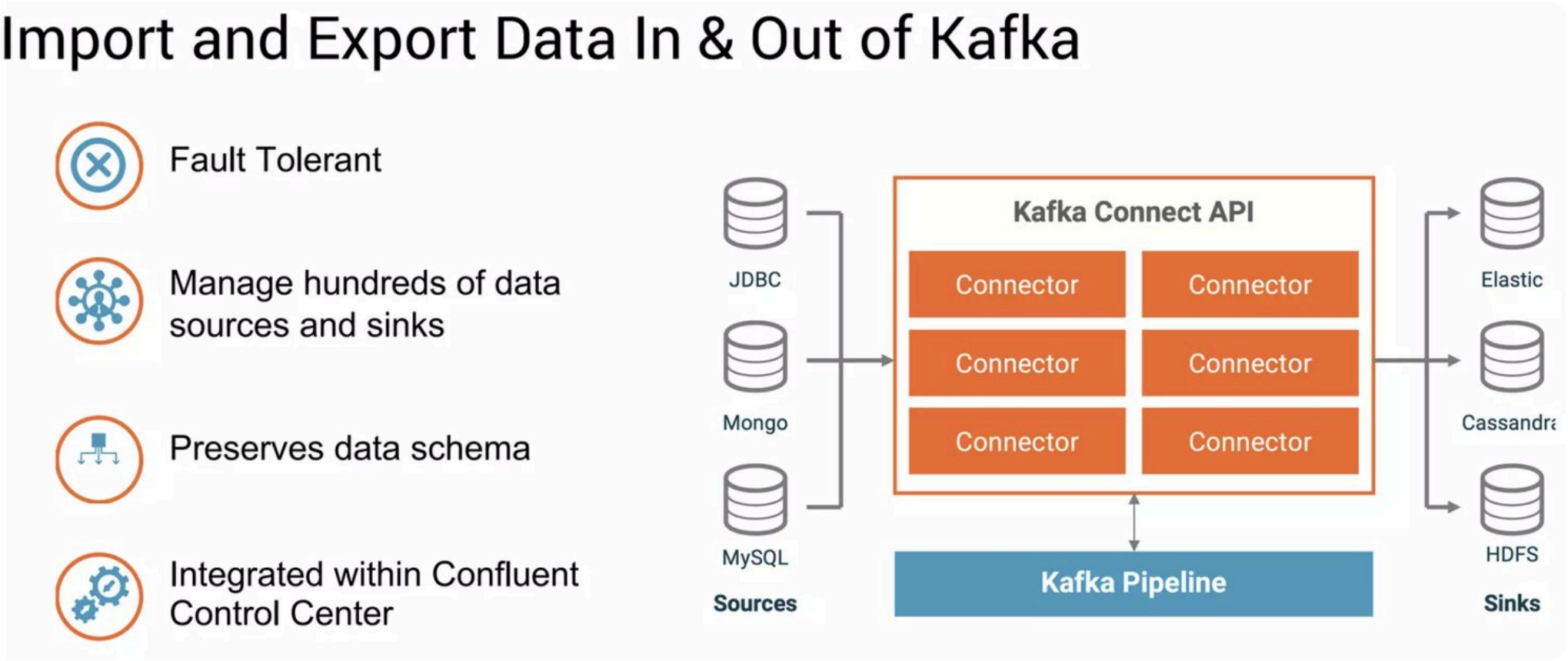


Add producers, brokers or consumers independently: **that is the point**

Programming in Kafka

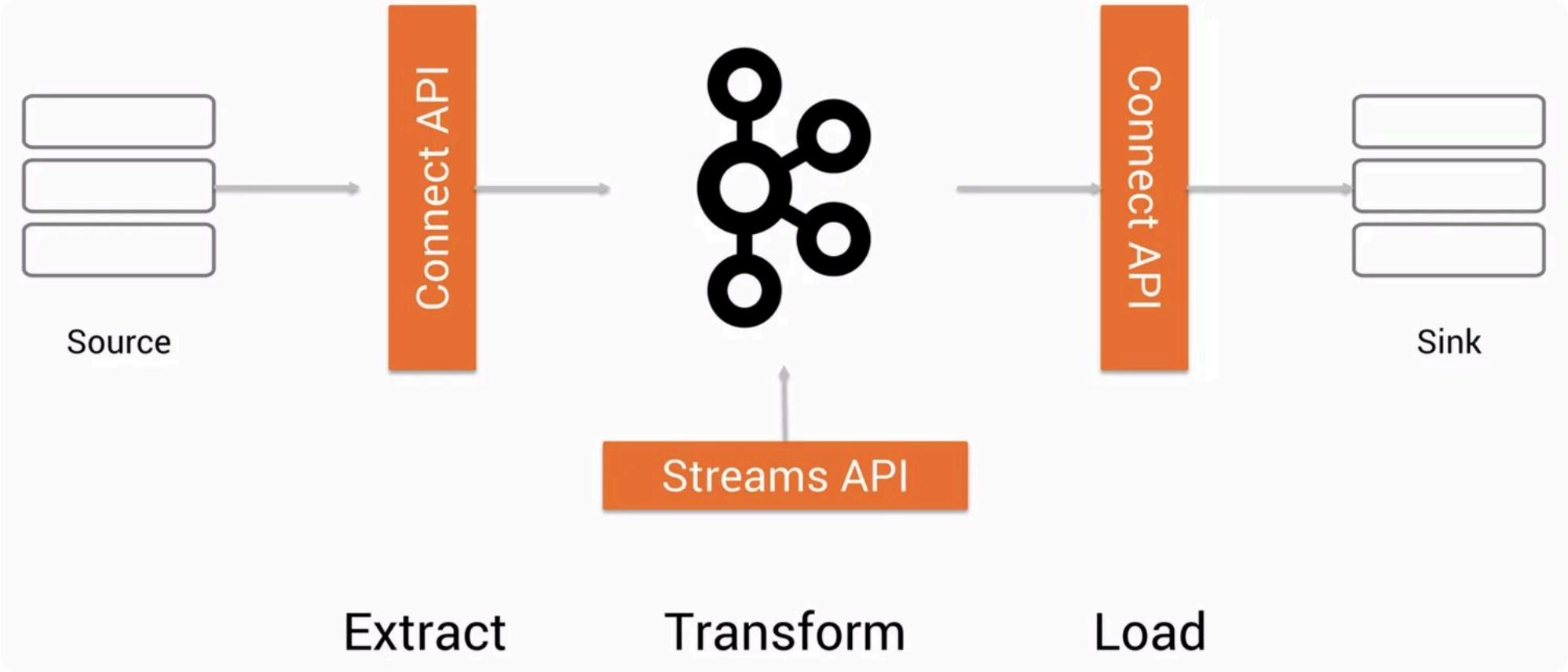
How do I get streams of data into and out of Kafka?

Kafka Connect Sources and Sinks



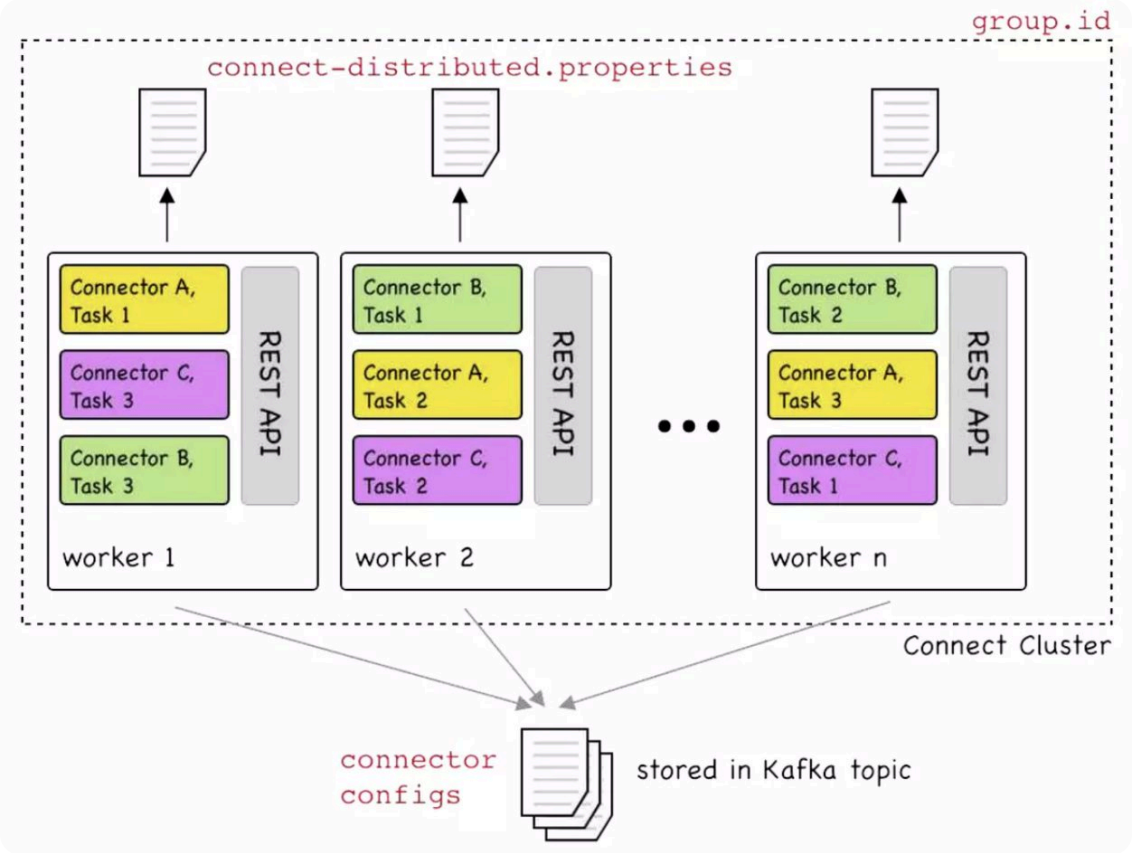
Source connectors in, sink connectors out: **configuration, not code**

A Continuous ETL Flow



Extract (source connector), transform (Streams), load (sink connector)

Kafka Connect Workers



Connector tasks spread over workers; configs live **in a Kafka topic**

Change Data Capture

Step	What happens	Typical tool
1. A row changes	an app runs INSERT, UPDATE or DELETE on its database	PostgreSQL, MySQL
2. The change is read	from the database log, not by re-querying the table	Debezium on Kafka Connect
3. It becomes an event	one message per change, in order, on a topic	Kafka
4. Others react	warehouse, search index, cache stay in sync	sink connectors, Flink

Read the database log and send **every change** as an event

Kafka Streams as a Library

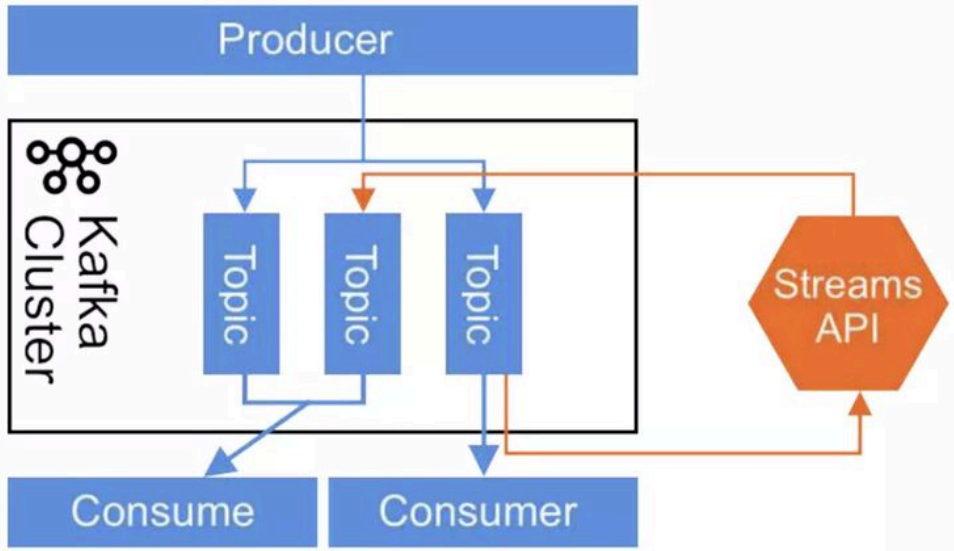
Transform Data with Real-Time Applications

Overview

- Write standard Java applications
- No separate processing cluster required
- Exactly-once processing semantics
- Elastic, highly scalable, fault-tolerant
- Fully integrated with Kafka security

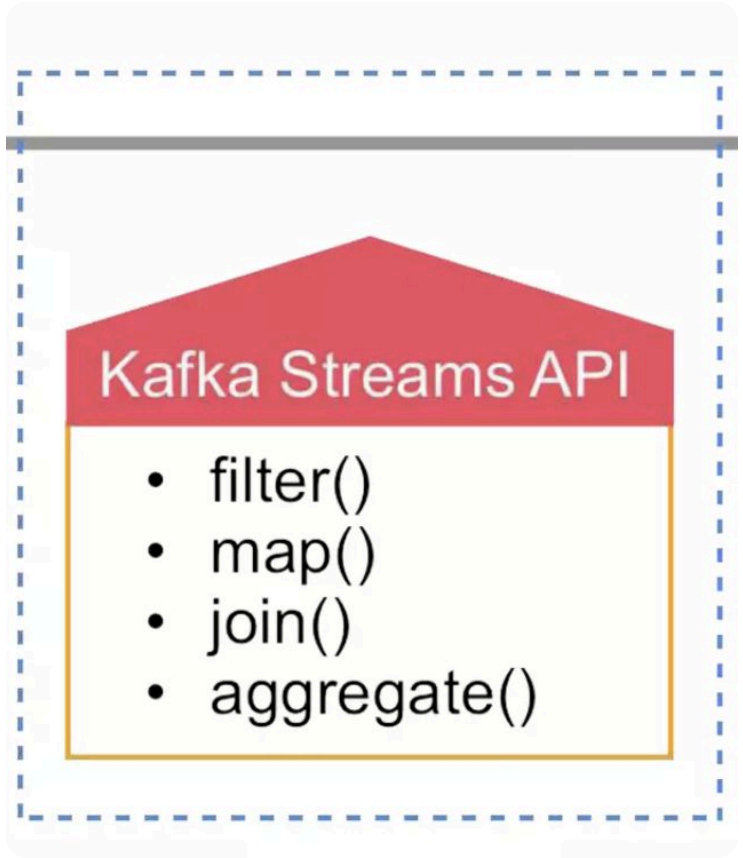
Example Use Cases

- Microservices
- Continuous queries
- Continuous transformations



A library, not a cluster: your app reads a topic, transforms, writes a topic

Continuous Transformations



filter, map, join, aggregate, **on an unbounded stream**

Confluent KSQL

Develop real-time stream processing apps writing only SQL!

No Java, Python, or other boilerplate to wrap around it.

```
CREATE STREAM vip_actions AS
  SELECT   userid, page, action
  FROM     clickstream c
  LEFT JOIN users u ON c.userid = u.user_id
  WHERE    u.level = 'Platinum';
```

Event Time vs Arrival Time

An event can arrive after the time window in which it happened

Event	Happened at	Arrived at	10:00–10:05 window
P-17 overheats	10:04	10:04	count it now
P-17 alarm	10:03	10:07	late for this window

The alarm belongs to the earlier window by **event time**

Watermarks and Late Events

A later close improves completeness but keeps window state longer

At 10:07	Policy	Result for the 10:00–10:05 window
Alarm from 10:03 arrives	close immediately at 10:05	miss the alarm
Alarm from 10:03 arrives	wait for late events until 10:08	revise the count

Latency and completeness move together when events arrive late

Stream Processing Engines

Engine	Runs as	Good at
Kafka Streams	a library inside your app	small transforms next to Kafka
Apache Flink	its own cluster	large stateful jobs, event-time windows, exactly-once
Spark Structured Streaming	Spark, in small batches	teams that already run Spark

Kafka moves the events; **Kafka Streams, Flink or Spark** compute on them

Example – Create Topic

Before you can write your first events, you must create a topic. Open another terminal session and run:

```
$ bin/kafka-topics.sh --create --topic quickstart-events --bootstrap-server
localhost:9092
```

All of Kafka's command line tools have additional options. Run `kafka-topics.sh` without arguments to display usage information, or describe the new topic:

```
$ bin/kafka-topics.sh --describe --topic quickstart-events --bootstrap-server
localhost:9092
Topic: quickstart-events   TopicId: NPmZHyhbR9y00wWglMH2sg   PartitionCount: 1
ReplicationFactor: 1
  Topic: quickstart-events   Partition: 0   Leader: 0   Replicas: 0   Isr: 0
```

Example – Process Events with Kafka Streams

To give you a first taste, here's how one would implement the popular `WordCount` algorithm:

```
KStream<String, String> textLines = builder.stream("quickstart-events");

KTable<String, Long> wordCounts = textLines
            .flatMapValues(line -> Arrays.asList(line.toLowerCase().split(" ")))
            .groupBy((keyIgnored, word) -> word)
            .count();

wordCounts.toStream().to("output-topic", Produced.with(Serdes.String(),
Serdes.Long()));
```

The same WordCount as Lecture 11's MapReduce and Spark, now **updated with every new line**: a KTable is a count that never finishes.

Takeaway: Big Data III

1. Kafka stores ongoing events in **partitioned logs**; order holds within a partition
2. Consumer groups scale reading, but a crash before an **offset commit** can replay events
3. Connect moves events; a stream processor computes by **event time** and must handle late arrivals